Ajaia Docs — Document Management System Design Deep Dive

Based on your `feat/documents` implementation log · Architecture, API design, permissions, database operations, concurrency, tradeoffs and testing

This time, we're studying what you actually built, rather than the earlier proposed design.

Your implementation includes seven API operations, centralized permission rules, optimistic concurrency, validation, sharing and database-level authorization checks. Your log reports 59 passing unit tests and 24 successful manual API scenarios.

This guide covers what each component does, why it was chosen, available alternatives, and potential failure cases.

1. What are we building?

System requirements:

> Design the backend for a Google Docs-like application where users can create, open, rename, edit, delete and share documents. Some users can edit, while others can only view. Prevent unauthorized access and accidental overwriting of another user's work.

The requirements are:

Functional requirements	Nonfunctional requirements
Create documents	Persistent storage
List owned and shared documents	Consistent permission enforcement
Open documents	Prevent unauthorized writes
Rename and edit documents	Prevent stale content overwrites
Delete documents	Validate incoming data
Share and revoke access	Return predictable API errors
Support Owner, Editor and Viewer	Testability and maintainability

Key design decision: We built a modular monolith with Next.js API routes, Prisma and PostgreSQL, rather than separate backend microservices.

That is appropriate for the size and time constraints of this project.

2. High-level architecture

React Frontend

Dashboard · Document editor · Share dialog

Next.js API Routes

Document and sharing endpoints

Session & CSRF

Identify user, check origin

Zod Validation

Validate IDs, title, JSON, roles

Access Rules

Owner, Editor, Viewer

Document Logic

Version checks, mutations

Prisma ORM

Queries · Filters · Atomic updates

Neon PostgreSQL

users · documents · document_shares

Why separate the code into different files?

Your implementation has these modules:

File	Responsibility	Why separate it?
`src/lib/access.ts`	Permission matrix and pure authorization functions	One consistent set of rules
`src/lib/validation.ts`	Zod schemas	Prevent invalid data reaching the database
`src/lib/documents.ts`	Database role lookup and editable-by filter	Reuse database access rules
`src/lib/http.ts`	JSON parsing, CSRF checks and error handling	Consistent HTTP behavior
API route handlers	Request orchestration	Keep endpoints focused

Alternatives considered

Approach	Advantages	Disadvantages
All logic in route handlers	Fastest initial implementation	Duplicated rules, difficult testing
Separate shared modules — chosen	Reusable, testable, maintainable	More files and abstractions
Separate document microservice	Independent deployment and scaling	Networking, service authentication, deployment complexity

Design rationale: We kept a single deployable backend but separated responsibilities internally. That gives us maintainability without introducing distributed-system complexity.

3. Authorization design

Authentication identifies the caller. Authorization determines what that caller can do with a document.

Our authorization model has four effective roles:

Action	Owner	Editor	Viewer	No access
View	Allow	Allow	Allow	404
Edit / rename	Allow	Allow	403	404
Share	Allow	403	403	404
Delete	Allow	403	403	404

The `OWNER` role comes from `documents.owner_id`. `EDITOR` and `VIEWER` come from `document_shares.role`.

Why use a permission matrix?

Alternative 1: Hardcode conditions throughout the endpoints.

```
if (role === "OWNER" || role === "EDITOR") {
  // allow editing
}
```

This works, but if five endpoints contain their own permission logic, they can drift apart.

Alternative 2: Centralize permissions.

```
const PERMISSIONS = {
  OWNER:  ["view", "edit", "share", "delete"],
  EDITOR: ["view", "edit"],
  VIEWER: ["view"]
};
```

Your implementation uses the second approach.

It provides three reusable operations:

- `getRole()` determines the effective role.
- `can()` checks whether an action is permitted.
- `assertCan()` rejects unauthorized actions.

These are pure functions in `access.ts`, which means the permission rules can be tested without a database.

Why 404 instead of 403 for outsiders?

Imagine Dave guesses the URL of Alice's private document.

If the API returns `403`, Dave learns that the document probably exists.

If it returns `404`, the application does not reveal whether the document exists or is private.

However, if Carol is already a viewer and attempts to edit, `403` is appropriate because Carol is allowed to know the document exists.

Tradeoff: Returning 404 for unauthorized resources makes debugging slightly less explicit, but protects document existence information.

4. Input validation

Your implementation uses Zod to validate data before database operations.

Input	Actual validation
Document title	Trimmed, 1–200 characters
Document ID	Valid UUID
Editor content	Object with `type: "doc"` and child-node array
Content size	Maximum 500 KB
Share email	Trimmed, lowercased, valid email
Share role	`VIEWER` or `EDITOR`, defaults to `VIEWER`
Content update	Requires document version
Empty PATCH	Rejected

Why validate in the API if PostgreSQL has constraints?

Because they protect different boundaries.

Zod checks the incoming request and produces useful error messages.

PostgreSQL protects stored data and relationships, even when application logic makes a mistake.

For example, PostgreSQL can enforce a foreign key, but it does not automatically understand whether arbitrary JSON represents a usable TipTap document.

Why the 500 KB limit?

Without a size limit, someone could send an extremely large JSON payload.

That could increase:

- Request processing time.
- Database storage and write costs.
- Memory consumption.
- Editor loading time.

The 500 KB value is a practical application limit, not a universal PostgreSQL limitation.

Tradeoff: Very large documents will be rejected. If supporting large documents becomes a requirement, we'd measure typical sizes and reconsider the storage and editing model.

Your current validator checks the essential JSON structure, not every possible TipTap node and mark. That's a documented limitation.

5. Document API design

Your implementation exposes seven method-and-path combinations.

Method	Endpoint	Purpose
GET	`/api/documents`	List owned and shared documents
POST	`/api/documents`	Create document
GET	`/api/documents/:id`	Open document
PATCH	`/api/documents/:id`	Rename or save content
DELETE	`/api/documents/:id`	Delete document
POST	`/api/documents/:id/shares`	Grant or update sharing
DELETE	`/api/documents/:id/shares/:userId`	Revoke sharing

Why REST instead of GraphQL?

REST — chosen	GraphQL
Straightforward resource endpoints	Flexible client-selected data
Simple HTTP status codes	Commonly returns operation details in a structured response

REST — chosen	GraphQL
Less infrastructure for a small app	Additional schema and resolver layer
Easy to test with curl	Useful for complex, nested data needs

REST is sufficient because our application has a small number of well-defined resources and operations.

6. Creating a document

Let’s trace the full request.

1. Alice clicks New Document.
2. Browser sends `POST /api/documents`.
3. Backend checks the session and request origin.
4. Zod validates the optional title and content.
5. Backend sets `ownerId` from Alice’s session.
6. Prisma inserts the document in PostgreSQL.
7. API returns `201 Created` with the new document.

Illustrative SQL:

```
INSERT INTO documents (
    owner_id,
    title,
    content
)
VALUES (
    $1,
    'Untitled document',
    $2::jsonb
)
RETURNING *;
```

The critical design decision is that `owner_id` comes from the server session.

Why not accept ownerId from the frontend?

Because Bob could send:

```
{
  "ownerId": "alice-user-id",
  "title": "Fake document"
}
```

The backend must not trust that identity claim.

7. Listing owned and shared documents

The dashboard needs two lists:

- Documents Alice owns.
- Documents other users have shared with Alice.

Your implementation runs two independent database queries in parallel using `Promise.all`.

Owned documents

```
SELECT id, title, updated_at
FROM documents
WHERE owner_id = $1
ORDER BY updated_at DESC;
```

Shared documents

```
SELECT
    d.id,
    d.title,
    d.updated_at,
    s.role,
    u.name AS owner_name,
    u.email AS owner_email
FROM document_shares s
JOIN documents d
    ON d.id = s.document_id
JOIN users u
    ON u.id = d.owner_id
WHERE s.user_id = $1
ORDER BY d.updated_at DESC;
```

Important tradeoffs

Decision	Why
Two lists instead of one	UI clearly distinguishes owned and shared
Fetch metadata only	Avoid transferring full document JSON
Run queries in parallel	Reduce waiting when queries are independent
Order by `updatedAt DESC`	Show recently changed documents first
Include owner details	Users know who shared a document
Include share role	UI can show Edit or View-only access

Why not fetch document content?

Suppose Alice has 100 documents, each containing 100 KB of JSON.

Fetching every document’s content would transfer approximately 10 MB before she even opens one.

The dashboard only needs titles, timestamps and access information.

This is database projection: select only the fields required by the current operation.

Limitation

Your current implementation does not paginate document lists.

That is acceptable for a small demo. For a larger application, cursor-based pagination would be worth considering.

8. Opening a document

Bob clicks a document Alice shared with him.

The backend performs:

1. Validate the document ID.
2. Identify Bob from the session.
3. Determine Bob’s role.
4. Reject the request if Bob has no access.
5. Fetch the full document.
6. Return its content, version, owner, collaborators and Bob’s role.

Illustrative access query:

```
SELECT d.*
FROM documents d
WHERE d.id = $1
  AND (
    d.owner_id = $2
    OR EXISTS (
      SELECT 1
      FROM document_shares s
      WHERE s.document_id = d.id
        AND s.user_id = $2
    )
  );
```

Why return the user’s role?

Because the frontend needs to know which controls to display.

An editor can see editing controls. A viewer gets a read-only interface. An owner can also manage sharing and deletion.

However, the frontend is not the security boundary. The backend still verifies every protected request.

9. Saving a document — optimistic concurrency

This is the most important technical decision in your implementation.

A document has a version number.

When a user opens a document, the API returns its current version.

When the user saves content, the request includes the version they originally loaded.

The database accepts the save only if the current version still matches.

The problem: Lost updates

Alice

Opens version 1

Edits heading

Saves → version 2

Bob

Opens version 1

Edits paragraph

Save → 409

Bob's save is rejected because his document version is outdated. Alice's saved content remains intact.

What alternatives did we have?

Approach	Advantages	Disadvantages
Last-write-wins	Simplest implementation	Silent lost updates
Pessimistic locking	Prevents conflicting writes while locked	Locks are unsuitable for long editing sessions
Optimistic concurrency — chosen	Prevents silent overwrites without long-lived locks	Requires conflict handling
Yjs / CRDT	Supports merging concurrent edits	Much greater complexity

How your implementation works

The client sends:

```
{
  "content": {
    "type": "doc",
    "content": []
  },
```

```
  "version": 1
}
```

The backend:

1. Validates the content and version.
2. Checks the user's edit permission.
3. Updates only if both permission and version match.
4. Atomically increments the version.
5. Returns the new version.
6. If no row was updated, determines whether access changed, the document disappeared or the version was stale.

Illustrative SQL matching the design:

```
UPDATE documents d
SET
    content = $1::jsonb,
    version = version + 1,
    updated_at = NOW()
WHERE d.id = $2
  AND d.version = $3
  AND (
    d.owner_id = $4
    OR EXISTS (
      SELECT 1
      FROM document_shares s
      WHERE s.document_id = d.id
        AND s.user_id = $4
        AND s.role = 'EDITOR'
    )
  )
RETURNING d.version, d.updated_at;
```

Your Prisma implementation uses `updateMany` to apply the additional filters and inspect the affected row count.

Why `updateMany` instead of a simple update?

The implementation needs to express:

- Document ID matches.
- Version matches.
- User is owner or editor.

`updateMany` supports this conditional update pattern and reports how many records changed.

The crucial property is that checking the version and changing the version happen atomically in the database.

What happens when the version is stale?

The API returns:

`409 Conflict`

The save is rejected instead of overwriting the newer content.

A limitation remains: your backend does not automatically merge the conflicting changes. The frontend needs to handle the conflict, such as by asking the user to reload while preserving their unsaved draft.

10. Why check permissions twice?

Your implementation performs an initial authorization check and then includes permission conditions in the database mutation.

This is important because of a time-of-check to time-of-use (TOCTOU) race condition.

Consider this sequence:

1. Bob sends a save request.
2. Backend checks Bob's role: `EDITOR`.
3. Alice revokes Bob's access.
4. Bob's update reaches PostgreSQL.
5. Database re-evaluates the permission condition.
6. If the revocation is visible to the update, Bob's write affects zero rows.

Without the final condition, Bob could save based on an outdated permission decision.

Alternatives

Strategy	Tradeoff
Check permission only before UPDATE	Simpler, but creates a race window
Check before and inside UPDATE — chosen	Clear errors and stronger write-time enforcement
Use explicit transactions and locks	More control, but more complex

One subtle point: the conditional update protects against a revocation that commits before the update's authorization check. Fully defining behavior for truly simultaneous transactions requires considering PostgreSQL isolation and locking semantics.

Key takeaway: We enforce authorization at the mutation boundary rather than trusting a previous check.

11. Renaming a document

Your API supports both renaming and content updates through `PATCH /api/documents/:id`.

Rename request:

```
{
  "title": "Updated Project Proposal"
}
```

Unlike content saves, renames do not require a version.

Why?

Your implementation deliberately separates metadata changes from content concurrency.

If Alice renames a document while Bob edits its content, Bob's content save should not necessarily fail just because the title changed.

Design	Advantage	Disadvantage
Require version for every PATCH	Stronger conflict detection across all fields	Unnecessary conflicts between title and content
Require version only for content — chosen	Independent rename and content operations	Concurrent renames can overwrite each other
Separate title and content versions	More precise concurrency	Additional state and complexity

This is a reasonable take-home tradeoff.

If title collaboration became important, we could add a separate metadata version or use field-specific concurrency control.

12. Sharing documents

Alice shares a document with Bob using his email address.

The API receives:

```
{
  "email": "bob@ajaia.dev",
  "role": "EDITOR"
}
```

The backend:

1. Verifies Alice's session and request origin.
2. Validates and normalizes the email.
3. Checks that Alice owns the document.
4. Rejects sharing with herself.
5. Finds Bob's existing user record.
6. Creates or updates Bob's share.

7. Returns the share details.

Why use upsert?

Suppose Bob already has `VIEWER` access.

Alice changes him to `EDITOR`.

Without upsert, inserting another share could violate the unique `(document_id, user_id)` constraint.

With upsert, we can create a new record or update the existing role.

Illustrative SQL:

```
INSERT INTO document_shares (
    document_id,
    user_id,
    role
)
VALUES ($1, $2, 'EDITOR')
ON CONFLICT (document_id, user_id)
DO UPDATE SET role = EXCLUDED.role;
```

This SQL illustrates upsert behavior; the route must still enforce owner-only authorization, including protection against permission races where necessary.

Alternatives

Approach	Advantages	Disadvantages
Insert only	Very simple	Re-sharing produces duplicate errors
Delete then insert	Straightforward replacement	Extra writes and possible gap
Upsert — chosen	Create or update in one operation	Must carefully enforce authorization
Invitation system	Supports users without accounts	Email delivery and token lifecycle

Your implementation shares only with existing users. It does not send email invitations.

The default share role is `VIEWER`, following the principle of least privilege.

13. Revoking access

Alice removes Carol from a document.

`DELETE /api/documents/:id/shares/:userId`

After successful revocation:

- Carol’s share record is removed.
- Carol can no longer open the document.
- The document disappears from Carol’s shared list.

- Subsequent authorized-content updates must fail if Carol no longer has edit access.

The API returns `204 No Content`.

Important tradeoff: Revocation blocks future protected API access; it does not erase copies of content already downloaded to Carol's browser or device.

14. Deleting a document

Only the owner can delete a document.

Your implementation uses an owner-filtered delete:

```
DELETE FROM documents
WHERE id = $1
  AND owner_id = $2;
```

Because the sharing table has a foreign key with `ON DELETE CASCADE`, deleting the document also removes its shares.

Hard delete vs soft delete

Approach	Pros	Cons
Hard delete — chosen	Simple, immediate removal	No built-in recovery
Soft delete	Trash and restore support	More query filtering and lifecycle logic
Versioned deletion	Better audit/recovery	Additional data and retention management

For this assessment, hard delete is sufficient.

For production, a trash feature and retention policy would be worth considering.

15. Error handling

Your implementation uses consistent HTTP errors.

Scenario	Status
Missing session	`401`
Viewer tries editing	`403`
Outsider opens private document	`404`
Invalid title, JSON or share details	`400`
Stale document version	`409`
Successful creation	`201`

Scenario	Status
Successful deletion	`204`
Unexpected server failure	`500`

The shared HTTP helper also handles malformed JSON, origin checks and generic server errors.

Why not always return 500?

Because different errors require different frontend behavior.

A 400 means the client sent invalid data. A 403 means the user lacks permission. A 409 means the user should resolve a conflict rather than blindly retry.

Predictable error semantics make the API easier to integrate and debug.

16. Testing strategy

Your implementation log reports:

Unit tests

59

Passed

Manual API scenarios

24

Passed

The unit tests cover access rules, validation and HTTP helpers.

The manual tests exercised actual API routes with sessions and Neon PostgreSQL, including:

Scenario	Expected result
Signed-out list	401
Carol opens unshared document	404
Alice shares with Carol	201
Carol tries editing	403
Bob saves version 1	200, new version 2
Alice saves stale version 1	409
Bob tries deleting	403

Scenario	Expected result
Alice revokes Carol	204
Carol opens after revocation	404
Alice deletes document	204

Why both unit and manual tests?

Unit tests verify individual rules quickly.

Manual API tests verify that sessions, routes, validation, permissions and database operations work together.

However, these are not the same as a fully automated integration suite. Your log identifies database-level integration tests, including concurrency races, as future improvements.

17. What would we improve for production?

Area	Current implementation	Potential improvement
Authentication	Seeded-user demo sessions	Real identity provider
Concurrency	Version-based 409	Conflict-resolution UI or CRDT
Sharing	Existing users only	Email invitations and audit history
Validation	Essential TipTap JSON shape	Full editor-schema validation
Document lists	No pagination	Cursor pagination
Deletion	Hard delete	Trash and recovery
Testing	Unit tests + manual API tests	Automated database integration tests
Observability	Not established by this log	Structured logs, metrics and alerts
Large documents	500 KB content limit	Evaluate larger-content architecture

The goal is not to implement all of these immediately. Each improvement should respond to a real product requirement or observed limitation.

18. Quick Questions

1. Why did we use a modular monolith?

It keeps deployment simple while separating authentication, authorization, validation, and document logic into reusable modules. Microservices would add unnecessary complexity.

2. Why centralize permissions?

To enforce consistent Owner, Editor, and Viewer rules across all endpoints, avoid duplicated logic, and make authorization easier to test.

3. Why use optimistic concurrency?

To prevent users from accidentally overwriting each other's changes. Each save checks the document version and returns `409 Conflict` if it is outdated.

4. Why check authorization inside the UPDATE?

Permissions might change after the initial check. Including authorization conditions in the database update helps prevent writes based on outdated permissions.

5. Why use `updateMany`?

Prisma's `updateMany` allows checking document ID, version, and permissions in one atomic conditional update and reports whether a row was updated.

6. Why doesn't renaming require a version?

Renaming changes metadata, not document content. Keeping these operations independent avoids unnecessary conflicts, although simultaneous renames can overwrite each other.

7. Why use upsert for sharing?

Upsert creates a new share if none exists or updates the existing role, preventing duplicate records and simplifying role changes.

8. Why return 404 for an outsider?

Returning `404 Not Found` prevents unauthorized users from discovering whether a private document exists.

9. Why only load metadata on the dashboard?

The dashboard only needs titles, timestamps, owners, and roles. Avoiding full document content reduces database work, network transfer, and loading time.

10. What are the biggest limitations?

No automatic merging of concurrent edits, no pagination, sharing only with existing users, demo authentication, and no recovery after hard deletion.